

SwasthyaSetu Updates & Implementation History

Document Type: Architectural & Implementation Change Log

Current System State: Production Core Verified

Last Updated: September 10, 2026

Authoritative Location: `/updates.md` (Repository Root)

1. Project & Architecture Evolution

SwasthyaSetu was conceived as a digital public health bridge connecting vulnerable Indian citizens to central and state healthcare welfare schemes. Over multiple phases of engineering, the system evolved from early proof-of-concept prototypes into a hardened, production-ready distributed system:

1. Phase 1–4 (Core Services & Eligibility Foundation):
- Established the Fastify 5 backend daemon and Firebase Authentication integration.
 - Built the deterministic scheme evaluation engine (`rule-engine.ts`) with versioned criteria JSON.
 - Designed the initial Citizen and ASHA user interfaces.
2. Phase 5–8 (Caseload, Guidance & Assistance Workflows):
- Implemented frontline ASHA caseload tracking, follow-up scheduling, and field activity logs.
 - Built the healthcare access guidance roadmap and document readiness checklist engine.
 - Established the Citizen ↔ ASHA connection request and assistance request lifecycles.
3. Phase 9–10 (ASHA Intelligence & Leave Coverage):
- Developed deterministic proactive attention signals across assigned caseloads.
 - Built the ASHA leave request and automated temporary caseload delegation subsystem.
4. Phase 11 & Multilingual Voice (Exotel + Sarvam AI):
- Integrated Exotel carrier telephony with Fastify full-duplex WebSockets (`/api/v1/voice/stream`).
 - Integrated Sarvam AI Saaras STT (`saaras:v3`) and Bulbul TTS (`bulbul:v3`) for real-time 8kHz μ-law voice streaming.
 - Built trilingual natural language understanding in Kannada (`kn-IN`), Hindi (`hi-IN`), and English (`en-IN`).
5. NFC Hardening & Public Resolver (Phases 1–7):
- Built end-to-end NFC card provisioning, one-time token architecture, SHA-256 hash storage, and two-phase card rotation.
 - Built the public rate-limited NFC resolver and simplified bilingual `/nfc` portal.
 - Resolved physical Android Web NFC NTAG213 write capacity bottlenecks.
6. Citizen ↔ Household Re-Architecture (Latest Core Milestone):
- Shifted household creation exclusively to Citizens during first-time onboarding.
 - Enforced strict 1 Citizen ↔ 1 Household invariant with deterministic IDs (`hh_${ownerUid}`).
 - Unified household and head member creation into an atomic transaction.
 - Fully disabled and blocked ASHA household creation (`403 FORBIDDEN_ROLE`).

2. Authentication & User Management

Evolution & Hardening

- **Multi-Role Firebase Authentication:** Supports email/password and phone authentication for Citizens, ASHA workers, and Administrators.
- **Privileged Registration Secrets:** Prevents unauthorized registration as ASHA or Admin by requiring pre-validation against SHA-256 hashed server-side secrets (`ASHA_REGISTRATION_SECRET_HASH` , `ADMIN_REGISTRATION_SECRET_HASH`).
- **Statutory Data Consent Gate (`requireConsent`):** Added mandatory statutory consent gating. Users cannot access household or casework data until agreeing to terms at `/auth/consent` , recorded with immutable timestamps in `/users/{uid}/consent_history` .
- **Server-Derived Identity:** All user requests derive identity strictly from the verified Firebase ID Token (`request.user.uid`), completely preventing client UID spoofing.

3. Citizen & Household Architecture

The Previous Architecture & Its Flaws

In the initial system design:

1. Household creation and member addition were separate HTTP operations (`POST /households` followed by `POST /households/me/members`).
2. If the user experienced a network drop between the two requests, a partially configured household remained in the database with no members.
3. ASHA workers were permitted to field-register households (`POST /api/v1/asha/cases`), creating parallel ownership semantics and ambiguity regarding which citizen owned the household record.
4. Multiple households could theoretically be created under edge cases.

The New Implemented Architecture

To establish clean ownership and eliminate partial configurations, the system was re-architected:

1. **Citizen-Driven Creation:** Households are now created **exclusively** by Citizens during first-time onboarding.
2. **Strict 1 Citizen ↔ 1 Household Invariant:**
 - Enforced at both the backend service layer (`HouseholdService.createHousehold`) and repository level.
 - Any attempt to create a second household returns: `HTTP 409 Conflict` \quad `{code: } {"HOUSEHOLD_ALREADY_EXISTS"}`
3. **Deterministic Identifier:** Household document ID is deterministically set to `hh_${ownerUid}` , binding the household permanently to the authenticated citizen.
4. **Server-Enforced Ownership:** `ownerUid` is derived directly from the verified Firebase token. Client-supplied `ownerUid` properties are ignored.
5. **Role Protection:** `POST /api/v1/households` requires `requireRole(["CITIZEN"])` . ASHA and Admin tokens are rejected with `403 Forbidden` .
6. **Single Atomic Transaction:**

- Household creation accepts `headAge` and `headGender` alongside household profile data.
 - Both the household record and initial Head-of-Household member are persisted together via `createHouseholdWithMembers()` using a Firestore atomic batch write (`batch.commit()`).
 - If either write fails, the entire operation rolls back—guaranteeing zero orphaned records.
7. **Clean Portal State:** Once created, the Citizen Portal loads directly; all "Set Up Household" cards and creation prompts are permanently removed.

4. ASHA Architecture Changes

Deprecation of ASHA Household Registration

Frontline ASHA workers are community healthcare facilitators, not data entry gatekeepers. Allowing ASHA workers to create independent household accounts created duplicate data silos.

Modifications Implemented:

1. **UI Removal:** Removed all 3 "Register Household" buttons, register modal state, and register modal JSX from `frontend/app/asha/page.tsx`.
2. **API Endpoint Blocked:** `POST /api/v1/asha/cases` was disabled and configured to reject all requests with: `HTTP 403 Forbidden` with message: "ASHA healthcare workers can no longer register households directly..."
3. **Assigned Caseload Model:** ASHA workers now work exclusively with households assigned via:
 - Citizen Connection Request acceptance (`POST /v1/asha/connection-requests/:id/accept`).
 - Administrator assignment (`POST /v1/admin/cases/assign`).

Preserved ASHA Capabilities

All core ASHA workflows remain fully functional:

- **Caseload Directory & Detail Drawer:** Inspect household address, socio-economic category, family roster, and health indicators.
 - **Scheme Journey Tracking:** Multi-step journey milestones (`SchemeJourneyStep`) and task completion checklists.
 - **Healthcare Gaps Triage:** Deterministic detection of unregistered pregnancies, unimmunized infants, and senior citizen coverage gaps.
 - **Field Notes & Follow-Ups:** Timestamped field observations and home visit scheduling (`Due Today` , `Overdue` , `Upcoming` , `Completed`).
 - **NFC Card Management:** Physical card provisioning, writing, rotation, and revocation for assigned households.
 - **Outbound PSTN Telephony:** Triggering phone calls to households via Exotel carrier gateway.
 - **ASHA Leave & Caseload Delegation:** Submitting leave requests with automatic temporary delegation of active cases to a designated coverage worker.
-

5. NFC Implementation Evolution

Architecture & Security Model

1. Zero Sensitive Data on Physical Card:

- The physical NFC tag stores **ONLY** a minimal URL record: `https://nfc?hh=hh_&t=}`
- The card strictly contains **NO** names, phone numbers, Aadhaar numbers, medical conditions, case notes, documents, or UIDs.
- Physical possession of the card does **NOT** grant authenticated Citizen Portal access; it resolves only a privacy-minimal public summary.

2. One-Time Token & Hash Storage:

- A cryptographically secure 16-byte random token is generated during provisioning.
- The raw token is returned to the ASHA worker's device **once** to write to the physical card.
- The backend stores **ONLY** the SHA-256 hash (`tokenHash`) in `household_nfc`.

3. Rate-Limited Public Resolver (`POST /v1/nfc/resolve`):

- Timing-safe hash comparison (`crypto.timingSafeEqual`) prevents timing attacks.
- Client IP rate-limiting (10 attempts/min) prevents brute-force token enumeration.
- Returns strictly privacy-minimal public data: household display name, region (village, district, state), high-level eligible scheme names, and safe ASHA service contact. Excludes ration card numbers, income categories, phone numbers, and individual member rosters.

Physical Write Optimization & NTAG213 Bug Resolution

During live physical NFC testing on Android devices with NTAG213 tags, several real-world hardware and protocol issues were identified and resolved:

1. Issue 1: Fastify Empty Body Error during Provisioning:

- *Problem:* Fastify threw `400 Bad Request` ("Body cannot be empty when content-type is set to application/json") during `POST /v1/asha/households/:id/nfc`.
- *Fix:* Configured API client to omit `Content-Type: application/json` when the body is undefined, and updated `nfc-service.ts` to explicitly send `{}`.

2. Issue 2: Transient NFC Communication Interruptions:

- *Problem:* Android NFC hardware occasionally lost carrier coupling during write start, throwing `DOMException NetworkError`.
- *Fix:* Added `overwrite: true` to `NDEFReader.write()` and implemented an automatic single retry after a 250ms delay on transient `NetworkError`.

3. Issue 3: NTAG213 Storage Capacity Overflow (Decisive Hardware Fix):

- *Problem:* NTAG213 tags have 144 bytes total memory, with a usable NDEF storage limit of **117 bytes**. On Vercel preview/branch deployments, URLs generated with branch hostnames (e.g., `sih-final-frontend-git-branch...vercel.app`) exceeded 121 characters, producing a 118-byte NDEF payload—exceeding capacity by exactly 1 byte and triggering Android `ERROR_IO` ("Failed to write due to an IO error: null").
- *Fix 1:* Updated `getPublicOrigin()` in `nfc-writer.ts` to prioritize `window.location.origin` over long Vercel deployment URLs during browser execution.

- *Fix 2:* Updated `buildNfcUrl()` to omit the optional query parameter `&v=1` when `version === 1`, saving 4 bytes.
- *Result:* Real SwasthyaSetu NFC URLs fit comfortably under 105 bytes, writing reliably to NTAG213 tags on Android with zero IO errors.

6. Citizen Portal & Onboarding Evolution

- **Dedicated First-Time Onboarding Screen:** Replaced confusing empty dashboard states with a clean, single-screen onboarding workflow that collects address, socio-economic category, ration card number, and head-of-household details.
- **Unified Single-Action Setup:** Eliminates multi-step wizard drop-offs by submitting household and head member data in one atomic API request.
- **Seamless State Transition:** React state immediately updates upon successful creation, revealing the full Citizen Portal without page reloads.
- **Elimination of Duplicate Setup CTAs:** All redundant "Set Up Household" cards, banners, and modal triggers were removed once a household is linked.

7. Backend & API Changes

1. Repository Layer Abstraction:

- Implemented `HouseholdRepository.createHouseholdWithMembers()` supporting Firestore atomic batch writes and transactional rollback for test isolation.
- Centralized memory stores across all repositories for ultrafast, reliable unit testing.

2. Route Guards & Middleware:

- Strict application of `requireAuth`, `requireConsent`, and `requireRole(["CITIZEN" | "ASHA" | "ADMIN"])`.
- Domain error handler mapping (`handleHouseholdError`, `handleCaseError`, `handleNfcError`) returning structured JSON error codes (`HOUSEHOLD_ALREADY_EXISTS`, `FORBIDDEN_ROLE`, etc.).

3. Clean Contract Typing:

- Extended `shared/types/household.ts` with `headAge`, `headGender`, and `initialMembers`.
- Updated Zod validation in `shared/schemas/household.schema.ts`.

8. Database & Firebase Controlled Reset

Context & Methodology

To transition cleanly to the new Citizen ↔ Household architecture and eliminate test artifacts, a controlled reset was executed live on Google Cloud project `swasthyasetu-efd78` using `scripts/execute-clean-reset.ts --execute`.

Reference Data vs Runtime Data Separation

Preserved Official Reference Data (Intact across reset):

- **schemes** (6 documents, 8 versions):
 - `ab-pmjay` (2 versions)
 - `jsy` (2 versions)
 - `jssk` (1 version)
 - `pmmvy` (1 version)
 - `state-health-assurance` (1 version)
 - `ab-ark-karnataka` (1 version)
- **evidence** (10 documents):
 - Official government gazette notifications and policy guidelines.

Reset Operational Collections (Purged to exactly 0 documents):

- `households` : 0 documents
- `cases` : 0 documents (and all nested subcollections `activities` , `notes` , `follow_ups` , `tasks`)
- `household_nfc` : 0 documents
- `asha_connection_requests` : 0 documents
- `asha_assistance_requests` : 0 documents
- `asha_leave_requests` : 0 documents
- `asha_temporary_assignments` : 0 documents
- `asha_leave_audit_logs` : 0 documents
- `voice_sessions` : 0 documents
- `ai_intelligence_cache` : 0 documents
- `evidence_search_cache` : 0 documents
- `evidence_audit_logs` : 0 documents
- `consent_records` : 0 documents
- `users` : 0 documents

Firebase Authentication Accounts:

- Purged all test accounts (including `asha@gmail.com`).
- Remaining accounts: 0.

9. Security & Privacy Improvements

1. Server-Enforced Ownership:

- Household `ownerUid` derived strictly from authentic JWT `request.user.uid` .
- Citizens cannot read, update, or delete households belonging to other users (tested and verified).

2. ASHA Caseload IDOR Defense:

- ASHA workers can only access cases assigned to their UID or temporarily delegated via approved leave.
- Unauthorized case access returns `404 Not Found` (tested and verified).

3. Cryptographic Token Verification:

- Constant-time verification (`crypto.timingSafeEqual`) on NFC token resolution.
- Rate-limiting (10 attempts/min) on public endpoints.

4. Secret Isolation:

- Zero exposure of third-party API keys (`SARVAM_API_KEY` , `EXOTEL_API_TOKEN` , `FIREBASE_PRIVATE_KEY` , `TAVILY_API_KEY`) to clients or public telemetry.

10. Testing & Verification Milestone

The current codebase state has been comprehensively verified:

1. Automated Backend Test Suite

```
npm test (in backend)
```

- **54 test files passed (54/54)**
- **689 total tests passed (689/689)**
- Test execution time: ~5.38 seconds.
- Specific suites verified:
 - `tests/household.test.ts` : 15/15 passed (Atomic onboarding, 1:1 invariant, 409 duplicate conflict, 403 non-citizen guard, transactional rollback).
 - `tests/case-routes.test.ts` : 7/7 passed (403 forbidden on ASHA registration, assigned case retrieval, IDOR defense, admin assignment).
 - `tests/case-service.test.ts` : 12/12 passed (403 rejection on field enrollment).
 - `tests/nfc*.test.ts` : 8 test files, 179/179 tests passed (Provisioning, rate-limited public resolver, two-phase rotation, simplified view, integration).
 - `tests/leave*.test.ts` : 100% passed (Leave creation, approval, temporary delegation, expiration restoration).
 - `tests/phase-f-multilingual-voice.test.ts` : 19/19 passed (Exotel WebSocket audio streaming, Sarvam STT/TTS in 3 languages, G.711 μ -law transcoding).

2. Frontend Production Build

```
npm run build (in frontend)
```

- Compiled successfully with Next.js 16.3.3 (Turbopack).
- TypeScript check passed with **0 errors**.
- All 12 static routes generated cleanly:
 - `/`
 - `/_not-found`
 - `/admin`
 - `/asha`
 - `/auth/consent`

- `/auth/sign-in`
- `/citizen`
- `/icon.png`
- `/nfc`
- `/unauthorized`

11. Deployment & Git History

Key Chronological Implementation Commits

Commit Hash	Commit Message / Description	Scope
1526d08	feat(architecture): atomic citizen onboarding, 1:1 household invariant, and disable asha household creation	Architecture
460f4f6	fix(nfc): optimize NDEF URL payload for NTAG213 capacity and prioritize client origin	NFC / Hardware
54809e3	fix(nfc): add single retry on transient NetworkError and improve NFC write resilience	NFC / Hardware
2805a22	fix(api): prevent empty JSON body Fastify error during ASHA NFC provisioning	Backend / API
9ee06dc	fix(nfc): isolate ASHA Register NFC click, harden modal, and add public NFC entry points	Frontend / NFC
2b8ae49	Final security audit	Security / Audit
d367952	Phase 6: Existing System Integration	Core Services
93d0372	Phase 5: Simplified Household View	Frontend / NFC
2f1b215	Phase 4: Detection + Retrieval	NFC Subsystem
c23e97f	Phase 3: NFC hardening	NFC Subsystem
12283bd	Phase 2: ASHA NFC provisioning	NFC Subsystem
46af587	Phase 1: NFC backend foundation	Backend / NFC
d98ddbfb	feat: add ASHA leave coverage workflow	ASHA Workflows

12. Current System State Snapshot

As of September 10, 2026, SwasthyaSetu operates as a fully integrated, verified healthcare access system:

- **Citizens:** Can register, submit consent, create their household and head member atomically in one step, manage family members, evaluate scheme eligibility deterministically, view guidance action plans, connect with local ASHA workers using service codes, request doorstep assistance, and request telephone callbacks.
- **ASHA Workers:** Can access assigned community caseloads, triage urgent health gaps, track scheme journey milestones, schedule home visits, record field notes, provision/rotate physical NFC cards, and request leave with automated caseload coverage. ASHA household creation is completely disabled.
- **Administrators:** Can supervise platform caseloads, reassign cases between workers, approve leave requests, review versioned schemes and evidence citations, and monitor real-time system telemetry.
- **NFC Physical Cards:** Fully compatible with NTAG213 tags, resolving privacy-minimal public benefit summaries via Web NFC and smartphones.
- **Carrier Telephony:** Fully operational over Exotel and Sarvam AI, supporting Kannada, Hindi, and English voice calls for non-smartphone populations.